

Group ICAC — AI Voice Coach

Port: 7869 | **GPU:** `gpu:a16:2` | **CPU:** 16 | **Mem:** 60 G | **Time:** 8 h | **Nginx path:** `/icac/`

Models: faster-whisper large-v3, `Qwen/Qwen2-VL-7B-Instruct` (4-bit, 2 GPUs), Kokoro TTS

Pipeline: audio → Whisper → Qwen coaching → Kokoro spoken feedback

Project Context

Course: INT6138 — Project II: Deploying Personal LLMs / ELMs, EdUHK **Cluster:** `aaillm.eduhk.hk` | **Account:** `student40` | **Hardware:** 4× NVIDIA A16 (16 GB VRAM each) **Job portal:** <https://aaillm.eduhk.hk> (Open OnDemand — SLURM job submission, file browser, JupyterLab) **Model cache:** `/opt/shared/model-cache` — all models pre-downloaded, `HF_HUB_OFFLINE=1` always set **Full architecture explanation:** [README.md](#)

How Your App Reaches the Browser

```
Browser
| https://aaillm.eduhk.hk/icac/
▼
nginx (port 443, TLS 1.2/1.3)
| location /icac/ → strips prefix → proxy_pass http://127.0.0.1:7869/
▼
Gradio app (0.0.0.0:7869) ← app.py inside a SLURM job (2× A16)
| demo.launch(server_name="0.0.0.0", server_port=7869, root_path="/icac")
▼
Python functions (transcribe with Whisper, coach with Qwen2-VL-7B, speak with
Kokoro TTS)
▼
2× NVIDIA A16 GPUs ← allocated by: sbatch submit_app.slurm
```

nginx location block (from `reverse-proxy.conf`)

```
# Group ICAC — port 7869
location /icac/ {
    proxy_pass          http://127.0.0.1:7869/;
    proxy_set_header    Host      $host;
    proxy_set_header    X-Real-IP $remote_addr;
    proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header    X-Forwarded-Proto $scheme;
    proxy_http_version  1.1;
    proxy_set_header    Upgrade    $http_upgrade;
    proxy_set_header    Connection "upgrade";
    proxy_read_timeout  3600s;
    proxy_send_timeout  3600s;
}
```

Key directives:

- `proxy_pass http://127.0.0.1:7869/` — trailing slash strips the `/icac/` prefix; Gradio receives plain `/` requests
 - `root_path="/icac"` in `demo.launch()` — tells Gradio to prefix all generated asset and API URLs with `/icac/`
 - `Upgrade $http_upgrade` — enables WebSocket; without this Gradio's real-time events (button clicks, streaming output) fail silently
 - `proxy_read_timeout 3600s` — critical for the full pipeline: Whisper transcription + Qwen2-VL coaching inference can take several minutes per session
-

submit_app.slurm

```
#!/bin/bash
#SBATCH --job-name=icac_app
#SBATCH --partition=gpu
#SBATCH --gres=gpu:a16:2
#SBATCH --cpus-per-task=16
#SBATCH --mem=60G
#SBATCH --time=08:00:00
#SBATCH --output=logs/%j_icac_app.out
#SBATCH --error=logs/%j_icac_app.err

source /opt/apps/python-envs/week07/bin/activate
export HF_HOME=/opt/shared/model-cache
export HF_HUB_OFFLINE=1
export GRADIO_SERVER_PORT=7869

mkdir -p logs
cd ~/ProjectCode/groups/Group_ICAC
python app.py
```

app.py

Imports & Config

```
import os, torch, tempfile, gradio as gr

os.environ.setdefault("HF_HOME", "/opt/shared/model-cache")
os.environ.setdefault("HF_HUB_OFFLINE", "1")
```

Model Loading (3-stage pipeline)

```

# Stage 1: Whisper large-v3
from faster_whisper import WhisperModel
whisper = WhisperModel("large-v3", device="cuda", compute_type="float16")

# Stage 2: Qwen2-VL-7B distributed across 2 GPUs
from transformers import Qwen2VLForConditionalGeneration, AutoProcessor,
BitsAndBytesConfig

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True, bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4"
)
processor = AutoProcessor.from_pretrained("Qwen/Qwen2-VL-7B-Instruct")
qwen_model = Qwen2VLForConditionalGeneration.from_pretrained(
    "Qwen/Qwen2-VL-7B-Instruct",
    quantization_config=bnb_config,
    device_map="auto",          # distributes layers across both A16s
    torch_dtype=torch.float16,
)
qwen_model.eval()

# Stage 3: Kokoro TTS (optional – app continues if not installed)
try:
    from kokoro import KPipeline
    kokoro_pipeline = KPipeline(lang_code="a")    # 'a' = American English
    KOKORO_AVAILABLE = True
except (ImportError, Exception):
    KOKORO_AVAILABLE = False
    kokoro_pipeline = None

```

Core Functions

```

COACHING_SYSTEM = (
    "You are an expert public speaking coach. Give specific, actionable, "
    "encouraging feedback. Focus on clarity, confidence, and structure. Under 100 "
    "words."
)
CANTONESE_SYSTEM = (
    "你係一位專業的普通話同英語演講教練。"
    "請用繁體中文提供具體、可行、鼓勵性的反饋。集中在表達清晰度、自信心同結構上。反饋不超 "
    "過100字。"
)

def _whisper_transcribe(audio_path, language):
    """Audio file → transcript string"""
    lang = None if language == "auto" else language
    segs, _ = whisper.transcribe(audio_path, language=lang, vad_filter=True,
    beam_size=5)
    return " ".join(s.text.strip() for s in segs)

```

```

def _qwen_coach(transcript, language):
    """Transcript + language code → coaching feedback string (Qwen2-VL-7B)"""
    system = CANTONESE_SYSTEM if language in ("zh", "yue") else COACHING_SYSTEM
    messages = [
        {"role": "system", "content": system},
        {"role": "user", "content": f'The speaker said:\n'
        {transcript}"\n\nPlease give coaching feedback.'},
    ]
    text_input = processor.apply_chat_template(messages, tokenize=False,
    add_generation_prompt=True)
    inputs = processor(text=[text_input], return_tensors="pt")
    inputs = {k: v.to(qwen_model.device) for k, v in inputs.items()}
    with torch.no_grad():
        out = qwen_model.generate(**inputs, max_new_tokens=200, temperature=0.6,
    do_sample=True)
    decoded = processor.decode(out[0], skip_special_tokens=True)
    return decoded.split("assistant")[-1].strip() if "assistant" in
    decoded.lower() else decoded.strip()

def _kokoro_speak(text):
    """Coaching text → spoken .wav file path (Kokoro TTS), or None if
    unavailable."""
    if not KOKORO_AVAILABLE or not text.strip():
        return None
    try:
        import soundfile as sf, numpy as np
        chunks = list(kokoro_pipeline(text, voice="af_heart", speed=1.0))
        if not chunks:
            return None
        audio_data = np.concatenate([c.numpy() if hasattr(c, "numpy") else c for c
    in chunks])
        tmp = tempfile.NamedTemporaryFile(suffix=".wav", delete=False)
        sf.write(tmp.name, audio_data, 24000)
        return tmp.name
    except Exception as e:
        print(f"Kokoro error: {e}")
        return None

def voice_coach(audio_path, language):
    """Full pipeline: audio → transcript → feedback → spoken feedback"""
    if audio_path is None:
        return "", "Upload an audio file first.", None, ""
    transcript = _whisper_transcribe(audio_path, language)
    if not transcript.strip():
        return "", "No speech detected.", None, ""
    feedback = _qwen_coach(transcript, language)
    spoken_fb = _kokoro_speak(feedback)
    status = f"Transcribed {len(transcript)} chars | Feedback: {len(feedback)}
    chars"
    return transcript, feedback, spoken_fb, status

```

```
def text_coach(text, language):
    """Text-only coaching path (skips Whisper)."""
    if not text.strip():
        return "Please enter your speech text.", None
    feedback = _qwen_coach(text.strip(), language)
    return feedback, _kokoro_speak(feedback)
```

Gradio UI & Launch

```
PORT = int(os.environ.get("GRADIO_SERVER_PORT", 7860))

with gr.Blocks(title="Group ICAC") as demo:
    lang_dd = gr.Dropdown(["auto", "en", "zh", "yue", "ja", "ko"], value="en",
label="Language")

    with gr.Tab("Voice Coach (Audio Pipeline)"):
        t1_audio = gr.Audio(type="filepath", label="Speech recording")
        t1_btn = gr.Button("Analyse & Coach", variant="primary")
        t1_status = gr.Textbox(lines=1, interactive=False)
        t1_trans = gr.Textbox(lines=4, interactive=False, label="Transcript")
        t1_fb = gr.Textbox(lines=6, interactive=False, label="Coaching
feedback")
        t1_spoken = gr.Audio(type="filepath", label="Spoken feedback")
        t1_btn.click(voice_coach, inputs=[t1_audio, lang_dd],
outputs=[t1_trans, t1_fb, t1_spoken, t1_status])

    with gr.Tab("Text Coach"):
        t2_text = gr.Textbox(lines=8, placeholder="Type what you would say...")
        t2_btn = gr.Button("Get Coaching Feedback", variant="primary")
        t2_fb = gr.Textbox(lines=6, interactive=False)
        t2_spoken = gr.Audio(type="filepath", label="Spoken feedback")
        t2_btn.click(text_coach, inputs=[t2_text, lang_dd], outputs=[t2_fb,
t2_spoken])

demo.launch(server_name="0.0.0.0", server_port=PORT, share=False,
root_path="/icac")
```